@Component vs @Service vs @Repository (Semantic vs Functional)

Spring provides three stereotype annotations for defining beans:

- @Component
- @Service
- @Repository

At a surface level, **all three do the same thing**:

They register a class as a Spring-managed bean.

But in a real Spring application, they are **not interchangeable**.

Spring differentiates them in **two important ways**:

1. **Semantic meaning** (what role the class plays in the system)
2. **Functional behavior** (what extra features Spring applies)

1. @Component — Generic Bean

@Component is the **base stereotype**.

It means:

"This class is a Spring-managed object."

It does **not** tell Spring:

- whether this class is business logic
- whether it talks to a database
- whether it needs transactions
- whether it needs persistence error handling

It simply says:

"Create an object and keep it in the container."

Example:

```
@Component
public class EmailValidator {
    public boolean isValid(String email) {
        return email.contains("@");
    }
}
```

@Component has **no special runtime behavior**.
It only makes the class eligible for dependency injection.

2. @Service — Business Logic Layer

@Service is a specialization of @Component.

It tells Spring:

"This class represents business logic."

This is **semantic** — but also **functional**.

Spring treats @Service beans as:

- Candidates for transaction management
- Targets for business-layer AOP
- Core application logic

Example:

```
@Service
public class UserService {
    public UserDTO getUser(Long id) {
        // business rules + orchestration
    }
}
```

Why Spring needs this:

- Business logic often needs:
    - Transactions
    - Security
    - Caching
    - Logging

Spring applies these cross-cutting concerns mainly around @Service beans.

So **@Service** is not just naming —
it marks the **heart of the application**.

3. @Repository — Data Access Layer

@Repository is also a specialization of @Component.

It tells Spring:

"This class talks to the database."

But it has a **very important extra behavior**:

Exception Translation

When a database error occurs, Spring will:

- Catch low-level exceptions (SQLException, HibernateException, etc.)
- Convert them into Spring's unified exceptions (DataAccessException)

This only happens for beans marked as @Repository.

Example:

```
@Repository
public interface UserRepository extends JpaRepository<User, Long> {
}
```

If you mark this as @Component instead:

- Spring **will not** translate database exceptions
- Your service layer will see raw persistence exceptions

So @Repository is **functionally different**.

Semantic vs Functional Difference

Annotation	Semantic Meaning	Functional Behavior
@Component	Generic Spring bean	No special behavior
@Service	Business logic	Transaction, AOP, orchestration
@Repository	Data access	Exception translation, persistence handling

Why Spring Doesn't Want You to Use @Component Everywhere

You *can* use @Component everywhere.
But then Spring cannot understand:

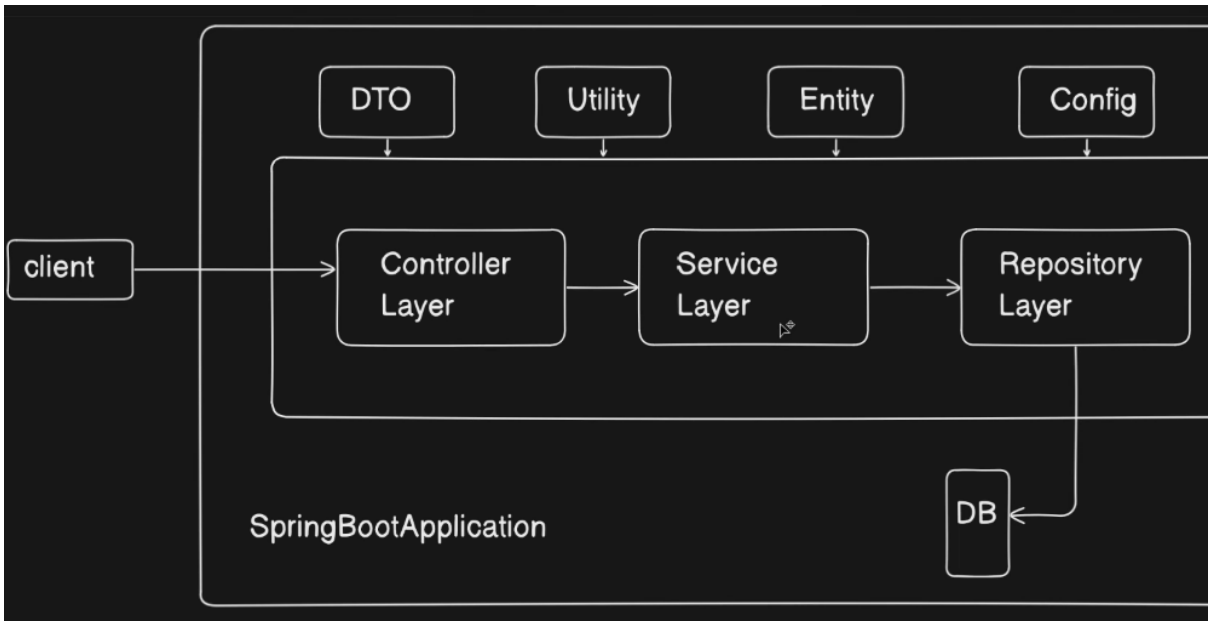
- Which class is business logic
- Which class is database access
- Where to apply transactions
- Where to translate exceptions

Spring's power comes from **knowing the role of each class**.

That's why it gives you:

- @Service for business logic
- @Repository for data access
- @Component for everything else

Complete Layered Architecture + Annotations



What Spring Boot Really Looks Like

A real backend application follows this flow:

```
Client
   ↓
Controller  (@RestController)
   ↓
Service     (@Service)
   ↓
Repository  (@Repository)
   ↓
Database
```

Spring enforces this through annotations.

Example — User Registration API

Let's build a simple:

API : GET /users/{id}

Entity (Database Model)

```
@Entity
@Table(name = "users")
public class User {

    @Id
    private Long id;
    private String name;
    private String email;
}
```

This maps to the DB table users.

DTO (What client sees)

```
public class UserDTO {
    private String name;
    private String email;
}
```

We never expose Entity directly.

Repository Layer

```
@Repository
public interface UserRepository extends JpaRepository<User, Long> {
}
```

Why @Repository?

Spring converts SQL errors into Spring exceptions automatically.

Service Layer

```java
@Service
public class UserService {

    @Autowired
    private UserRepository userRepository;

    public UserDTO getUser(Long id) {
        User user = userRepository.findById(id).orElseThrow();

        UserDTO dto = new UserDTO();
        dto.setName(user.getName());
        dto.setEmail(user.getEmail());

        return dto;
    }
}
```

This is where:

- Business rules
- DTO ↔ Entity conversion
- Validation
  live.

Controller Layer

```java
@RestController
@RequestMapping("/users")
public class UserController {

    @Autowired
    private UserService userService;

    @GetMapping("/{id}")
    public UserDTO getUser(@PathVariable Long id) {
        return userService.getUser(id);
    }
}
```

Controller:

- Talks to client
- Never touches database
- Never contains business logic

Where @Component Fits

```java
@Component
public class EmailValidator {
    public boolean isValid(String email) {
        return email.contains("@");
    }
}
```

Used for:

- Utility classes
- Helpers

- Mappers
- Validators

Not business logic.
Not database.

Why Spring Cares About These Annotations

Annotation	Purpose	Extra Power
@Component	Generic bean	None
@Service	Business logic	Transaction support
@Repository	Data access	Exception translation

Interview Gold

Q: Can we replace @Service with @Component?
Yes — but you lose semantic meaning and AOP benefits.

Q: Why @Repository?
Because Spring wraps database exceptions only for @Repository beans.

Q: Where should DTO conversion happen?
Service layer.

Summary

@Component registers a bean,
@Service represents business logic,
@Repository represents database access with special behavior.

Interview-Grade One-Liner

"@Component just registers a bean, @Service represents business logic, and @Repository represents data access with special persistence behavior like exception translation."